

파이프라인 단계별 Input / Output / 하는 일

1. Vertex Shader

Input: Vertex Data — 정점마다 다른 값들 (위치, 색상, 법선, 텍스처 좌표 등). Attribute로 들어옴.

하는 일: 정점 하나하나를 받아서 3D 월드 공간 좌표를 클립 공간 좌표로 변환. Model → View → Projection 행렬을 곱해서 `gl_Position`에 씀.

Output: 클립 공간으로 변환된 정점 위치 (`gl_Position`), 다음 단계로 넘길 데이터 (`out` 변수들 — 색상, 텍스처 좌표 등).

```
// input
layout (location = 0) in vec3 vPosition; // Attribute
layout (location = 1) in vec3 vColor;    // Attribute
uniform mat4 m_matrix;                  // Uniform
uniform mat4 v_matrix;
uniform mat4 proj_matrix;

// output
out vec3 Color; // Fragment Shader로 넘어감

void main() {
    Color = vColor;
    gl_Position = proj_matrix * v_matrix * m_matrix * vec4(vPosition, 1.0);
}
```

🌀 행렬 곱은 오른쪽부터 적용. `proj * view * model * pos` 순서로 써도 실제 적용은 model → view → proj.

2. Primitive Setup

Input: Vertex Shader가 변환한 정점들.

하는 일: 정점들을 삼각형으로 묶음. Edge를 만들고 Frustum 밖으로 나간 삼각형을 Clipping. 완전히 밖에 있는 삼각형은 제거, 부분적으로 걸친 삼각형은 잘라서 작은 삼각형으로 재구성.

Output: 클리핑된 삼각형 목록. 이 삼각형들만 다음 단계로 넘어감.

🌀 Clipping을 여기서 하는 이유는 성능. 보이지도 않는 삼각형을 래스터화하는 건 순수한 낭비라서 이 단계에서 미리 걸러냄.

3. Rasterization

Input: Primitive Setup을 거친 삼각형들 + 각 정점에 붙어있는 데이터 (`out` 변수들).

하는 일: 삼각형이 화면의 어떤 픽셀들을 덮는지 판별. 구체적으로는 삼각형의 Bounding Box를 구하고, 그 안의 픽셀 중심점(half-integer 좌표)에 대해 Point-in-Triangle 테스트를 병렬로 수행. 삼각형 안에 있다고 판정된 픽셀은 Fragment를 생성하고, 세 정점의 데이터를 Barycentric Coordinates로 보간해서 값을 채움.

Output: Fragment들. 각 Fragment는 화면의 특정 픽셀 위치 + 보간된 색상/텍스처 좌표/깊이값을 가짐.

🔗 Rasterization의 Output은 픽셀이 아니라 **Fragment**. Fragment는 "픽셀이 될 후보"이고, 최종 픽셀이 되려면 Fragment Shader와 Blending을 거쳐야 함.

🔗 Barycentric 보간: 픽셀 위치를 $b1 \cdot v1 + b2 \cdot v2 + b3 \cdot v3$ 로 표현할 때 ($b1+b2+b3=1$), 그 b값을 가중치로 색상·깊이·텍스처 좌표를 섞음. 꼭짓점에 가까울수록 그 꼭짓점의 색상이 강하게 반영됨.

4. Fragment Shader

Input: Rasterization이 만든 Fragment들 — 보간된 색상, 텍스처 좌표, 깊이값 등 (`in` 변수로 받음).

하는 일: 각 Fragment의 최종 색상을 계산. 텍스처 샘플링, 조명 계산, 그림자 처리 등 픽셀 단위의 모든 시각 효과가 여기서 이뤄짐.

Output: 각 Fragment의 최종 색상값 (`out vec4 FragColor`).

```
// input (Vertex Shader의 out이 보간되어 들어옴)
in vec3 Color;

// output
out vec4 FragColor;

void main() {
    FragColor = vec4(Color, 1.0);
}
```

🔗 Vertex Shader의 `out` 변수 이름과 Fragment Shader의 `in` 변수 이름이 같아야 연결됨. 이름이 다르면 컴파일은 되지만 데이터가 안 넘어옴.

5. Blending

Input: Fragment Shader가 계산한 Fragment들의 색상 + 깊이값.

하는 일: 같은 픽셀 위치에 Fragment가 여러이면 z-buffer로 깊이를 비교해서 가장 앞에 있는 것만 통과시킴 (Depth Test). 투명도가 있는 경우 Alpha Blending 수행.

Output: 최종 픽셀 색상. Framebuffer에 기록됨 → 화면에 출력.

🔗 z-buffer는 픽셀마다 현재 가장 앞에 있는 Fragment의 깊이를 저장하는 버퍼. 새 Fragment가 들어오면 z값을 비교해서 더 앞에 있으면 교체, 뒤에 있으면 버림.

데이터 전달 3가지

Attribute

정점마다 값이 다른 데이터. Vertex Shader의 입력으로만 사용됨.

VBO에 데이터를 올려두고, VAO로 읽는 방법을 등록해서 Vertex Shader가 정점을 처리할 때마다 자동으로 해당 정점의 값을 꺼내줌.

```
// Shader 선언
layout (location = 0) in vec3 vPosition;
layout (location = 1) in vec3 vColor;
```

```
// C++에서 location 직접 찾아서 1회성으로 넣는 경우
GLuint loc = glGetUniformLocation(rendering_program, "vPosition");
glVertexAttrib4fv(loc, vertex_positions);
```

🔗 Attribute는 **Vertex Shader에만** 들어감. Fragment Shader에는 Attribute가 없고, Vertex Shader의 **out** → Rasterization 보간 → Fragment Shader의 **in** 경로로 데이터가 넘어옴.

Uniform

드로우콜 동안 모든 정점/Fragment에 동일하게 적용되는 전역 변수. Vertex Shader, Fragment Shader 어디서든 선언해서 쓸 수 있음.

변화 행렬처럼 정점마다 달라지지 않는 값, 애니메이션 타이머, 조명 위치 등에 사용.

```
// Shader 선언
uniform mat4 RotationMatrix;
uniform vec4 vColor;
uniform float time;
```

```
// C++에서 전달
// 방법 1: 래퍼 함수 (강의 코드)
prog.setUniform("RotationMatrix", rotationMatrix);

// 방법 2: OpenGL 직접 호출
GLint loc = glGetUniformLocation(rendering_program, "RotationMatrix");
glUniformMatrix4fv(loc, 1, GL_FALSE, &rotationMatrix[0][0]);

// 스칼라/벡터
glUniform1f(loc, val); // float 1개
glUniform3f(loc, x, y, z); // float 3개
glUniform4fv(loc, count, ptr); // vec4 배열
```

🔗 glUniformMatrix4fv 의 Transpose 파라미터:

- GL_FALSE → column-major 그대로 (GLM 기본값 → 항상 이걸 씀)
- GL_TRUE → row-major로 해석 (전치해서 읽음)

🔗 Attribute vs Uniform 구분법: "정점마다 값이 달라야 하나?" → 달라야 하면 Attribute, 같아도 되면 Uniform. 위치·색상은 정점마다 다르니까 Attribute. 변환 행렬은 삼각형 전체에 동일하게 적용되니까 Uniform.

Buffer (VBO + VAO)

대량의 Attribute 데이터를 GPU 메모리에 올려두고 효율적으로 쓰는 방법. Attribute를 한 번에 하나씩 넘기는 `glVertexAttrib4fv` 와 달리, 모든 정점 데이터를 한꺼번에 GPU에 올려두고 드로우콜마다 재사용함.

VBO (Vertex Buffer Object): 실제 데이터가 저장되는 GPU 메모리 버퍼. 그냥 바이트 덩어리라서 이게 위치인지 색상인지 GPU는 모름.

VAO (Vertex Array Object): "이 VBO에서 어떻게 데이터를 읽어라"는 설정을 저장하는 객체. 어떤 location에 연결할지, 몇 바이트짜리인지, stride와 offset이 얼마인지 등.

```
// VBO: 데이터 GPU에 올리기
GLuint vbo;
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

// VAO: 읽기 방법 등록
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
// location=0, vec3(float 3개), 정규화 안 함, stride=0(백백), offset=0
glEnableVertexAttribArray(0);

// 드로우콜
glBindVertexArray(vao);           // 설정 복원
glDrawArrays(GL_TRIANGLES, 0, 3); // 그리기
```

🔗 세 가지 데이터 전달 방식 비교

	Attribute (직접)	Uniform	Buffer (VBO+VAO)
값 변화	정점마다 다름	드로우콜 전체 동일	정점마다 다름
데이터 위치	CPU → GPU 매번	CPU → GPU 매번	GPU 메모리에 상주
적합한 경우	정점 수가 적을 때	행렬, 상수값	정점 수가 많을 때
성능	낮음	보통	높음

🔗 실제로는 `glVertexAttrib4fv` 로 Attribute를 직접 넣는 경우는 거의 없고, 정점이 3개짜리 삼각형 하나라도 VBO + VAO 쓰는 게 표준. 데이터를 GPU 메모리에 올려두면 드로우콜마다 CPU-GPU 간 데이터 전송이 없어서 훨씬 빠름.